Universidade Federal do Pará
Instituto de Ciências Exatas e Naturais
Faculdade de Ciência da Computação

Professor: Nelson Cruz Sampaio Neto
Disciplina: Análise de Algoritmos (EN05174)
Turma: 01 (2025.4) - Vespertino
Grupo:

Alessandro Reali Lopes Silva - 202304940049;
Felipe Lisboa Brasil - 202404940029;
Gian Victor Gonçalves Figueiredo - 202404940017;
Heitor Lima e Silva - 202404940013.

Algoritmos de Busca e Ordenação

Análise de Algoritmos

Belém - PA
2026

Questão 01

```
TREE-SORT(A)
1.      T ← Árvore-Vazia
2.      Para i de 1 até A.comprimento faça:
3.              BST-INSERT(T, A[i])
4.      IN-ORDER-TREE-WALK(T.raiz)

BST-INSERT(T, z)
1.      y ← NULL
2.      x ← T.raiz
3.      Enquanto x ≠ NULL:
4.              y ← x
5.              Se z < x.chave faça:
6.                      x ← x.esquerda
7.              Senão faça:
8.                      x ← x.direita
9.      Se y = NULL faça:
10.             T.raiz <- z
11.     Senão se z < y.chave faça:
12.             y.esquerda ← z
13.     Senão faça:
14.             y.direita ← z

IN-ORDER-TREE-WALK(x)
1.      Se x ≠ NULL faça:
2.              IN-ORDER-TREE-WALK(x.esquerda)
3.              Imprima x.chave
4.              IN-ORDER-TREE-WALK(x.direita)
```

Para realizar uma análise rigorosa do pior e melhor caso para um conjunto de n dados, é fundamental definir como esses elementos estão inicialmente distribuídos, pois a ordem de inserção determina diretamente a altura da árvore. No pior caso, o conjunto de dados já se encontra em ordem crescente ou decrescente, o que resulta na construção de uma árvore degenerada, onde cada nó possui apenas um filho, assemelhando-se a uma lista encadeada. Nessa configuração, a cada nova inserção, o algoritmo é obrigado a percorrer e comparar o novo elemento com todos os nós já presentes, gerando um custo de inserção que cresce linearmente; ao somarmos as inserções de todos os n elementos, a complexidade resultante é $\Theta(n^2)$.

Já no melhor caso, os dados são inseridos em uma ordem que permite à árvore manter-se balanceada, distribuindo os ramos de forma equilibrada em ambos os lados da raiz. Em uma árvore balanceada, a altura é minimizada para log(n), pois um novo nível de profundidade só é iniciado quando todos os nós do nível anterior estão completamente preenchidos. Como o custo para inserir um único elemento em uma estrutura com essa altura é logarítmico e precisamos repetir esse processo para todos os n dados do conjunto, a complexidade total de tempo para a ordenação no melhor cenário é definida como $\Theta(n \times \log(n))$.

Questão 02

```
Linear-Search (A, n, v)
1.      Para i = 1 até n faça:
2.              Se A[i] = v então:
3.                      Retorne i
4.      Retorne NULL
```

Na função Linear-Search, o laço invariante indica que no início de cada laço, o elemento v não foi encontrado no subvetor A[1..i-1]. Na inicialização, com i = 1, o elemento v não foi encontrado no sub-autor A[1..i-1] que nesse caso será um vetor vazio e, de modo trivial, v não pertence à um vetor vazio. Na manutenção, para cada i, o v não existe em A[1..i-1] e se o elemento não for A[i] então ocorrerá uma nova iteração e se o vetor não existe em A[1..i-1] e não é A[i] então, para a iteração i+1, o elemento v não existe em A[1..i] e assim subsequentemente. No término, há duas formas de parar: Se A[i] = v, pois assim é garantido que, até i, v não pertence ao subvetor A[1..i-1], ou se i = n+1, acabando os elementos do vetor e concluindo que v não pertence ao subvetor A[1..n], que é igual ao vetor A, logo v não pertence a A.
Para o melhor caso, o elemento v precisaria estar em A[1], ou seja, na primeira posição, garantindo uma verificação e assim uma complexidade de $\Theta(1)$, ou seja, constante.
Para o pior caso, o elemento não pode pertencer ao vetor A, assim o algoritmo verificará todo o vetor, iterando n+1 vezes até perceber que o elemento não existe no vetor, possuindo uma complexidade de $\Theta(n)$, ou seja, linear.

Questão 03

```
RECURSIVE-BINARY-SEARCH(A, n, v)
1.      Se n = 1 então:
2.              Retorne n
3.      Se n > 1 então:
4.              meio ← n // 2
5.              direita ← A[meio+1 ; n]
6.              esquerda ← A[1 ; meio-1]
7.              Se v = A[meio] então:
8.                      Retorne meio
9.              Se v > A[meio] então:
10.                     n ← n - meio
11.                     Retorne RECURSIVE-BINARY-SEARCH(direita, n, v)
12.             Se v < A[meio] então:
13.                     n ← meio - 1
14.                     Retorne RECURSIVE-BINARY-SEARCH(esquerda, n, v)
```

Na função **Recursive-Binary-Search**, temos o caso base de uma lista unitária (n=1) na qual o elemento está na posição 1. Em seguida, realizamos a ideia de divisão e conquista por meio da partição da lista no lado direito esquerdo em relação ao valor do meio. A partir do tamanho da matriz, obtemos o tamanho intermediário e denominamos meio, em seguida, realizamos uma comparação do elemento da lista A[meio] com a entrada v, caso A[meio] coincida com o valor da entrada v, simplesmente retornamos a posição, pois o elemento foi encontrado. Caso v seja maior que meio, realizamos uma chamada recursiva utilizando como entrada a sublista Direita. Caso v seja menor que meio, realizamos uma chamada recursiva utilizando como entrada a sublista Esquerda. O algoritmo termina ao chegar no caso base definido.

```
ITERATIVE-BINARY-SEARCH(A, n, v)
1.      meio ←  n // 2
2.      esquerda ←  A[1; meio-1]
3.      direita ← A[meio+1; n]
4.      enquanto v ≠ A[meio] e n > 1 faça:
5.              Se v>A[meio] então:
6.                      A ← direita
7.                      n ← n-meio
8.                      meio ← n//2
9.                      esquerda ←  A[1; meio-1]
10.                     direita ← A[meio+1; n]
11.                     Retorne NULL
12.     Se v < A[meio] então:
13.             A ← esquerda
14.             n ← n-meio
15.             meio ← n // 2
16.             esquerda ←  A[1; meio-1]
17.             direita ← A[meio+1; n]
18.             Retorne NULL
19.     Se v=A[meio] então:
20.             Retorne meio
```

Na função **Iterative-Binary-Search**, inicia-se pela obtenção do valor intermediário da lista e das sublistas Esquerda e Direita. Em seguida, é utilizado um laço que tem como condição de parada a equivalência entre v e A[meio], além da garantia da lista ter mais de um elemento, i.e, n>1. Ao chegar ao final, é verificado se o valor armazenado na variável *meio* coincide com o valor v de busca.

Provaremos a complexidade do algoritmo a partir da análise assintótica a seguir:
Seja T(n) o número de instruções realizadas em função de uma dada entrada n, temos:

```
RECURSIVE-BINARY-SEARCH(A, n, v)

1.  Se n = 1 então:                     → Θ(1)
2.     Retorne n                        → Θ(1)
3.  Se n > 1 então:                     → Θ(1)
4.     meio ←  n // 2                   → Θ(1)
5.     direita ← A[meio+1 ; n]                  → Θ(1)
6.     esquerda ←  A[1 ; meio-1]         → Θ(1)
7.     Se v = A[meio] então:            → Θ(1)
8.        Retorne meio                  → Θ(1)
9.     Se v > A[meio] então:            → Θ(1)
10.       n ← n - meio                  → Θ(1)
11.       Retorne RECURSIVE-BINARY-SEARCH → Θ(T(n/2))
12.    Se v < A[meio] então:                    → Θ(1)
13.       n ← meio - 1                  → Θ(1)
14.       Retorne RECURSIVE-BINARY-SEARCH → Θ(T(n/2))
```

$$T(n) = \Theta(1) + \Theta(1) + \Theta(1) + \Theta(1) + \Theta(1) + \Theta(1) + \Theta(1) + \Theta(1) + \Theta(1) + \Theta(1) + \Theta(T(n/2))$$

Realizando as simplificações, Obtemos:

$$T(n)=\Theta(1),\ n=1$$
$$T(n) = \Theta(1) + \Theta(T(n/2)),\ n>1$$

Obtemos uma relação de recorrência. Resolvendo pelo teorema mestre, temos:

$$T(n) = T(n/2) + \Theta(1)$$

Separando os coeficientes, temos:

a = 1
b = 2
f(n) = Θ(1)

Em seguida, verificamos o n e comparamos com os casos:

n^(log_b a) = n^(log_2 1)
n^(log_2 1) = n^0
n^0 = 1
f(n) = Θ(1)
f(n) = Θ(n^(log_b a))

Implicando no caso 2 do Teorema Mestre, temos:

$$T(n) = \Theta(n^{(\log_b a)} \cdot \log n)$$
$$T(n) = \Theta(1 \cdot \log n)$$

Portanto,

$$T(n) = \Theta(\log n)$$

Questão 04

A substituição da busca linear pela busca binária no algoritmo **Insertion-Sort** não é capaz de reduzir a complexidade do pior caso para Θ(n log n), permanecendo o algoritmo com um tempo de execução de $\Theta(n^2)$. Embora a busca binária seja de fato mais eficiente, reduzindo o número de comparações necessárias para encontrar a posição de inserção de O(n) para O(log n), o algoritmo ainda enfrenta o gargalo estrutural da movimentação de dados em um array.
O verdadeiro problema reside no fato de que, uma vez encontrada a posição correta para a "chave", o algoritmo deve abrir espaço para inseri-la. Em um array, essa operação exige que todos os elementos à direita da posição de inserção sejam deslocados uma casa para a frente. No pior caso — quando o vetor está em ordem decrescente — cada nova inserção exige o deslocamento de todos os elementos já ordenados no sub-array. Portanto, mesmo que o local de inserção seja identificado rapidamente em tempo logarítmico, o custo para mover os elementos permanece linear O(n).
Dessa forma, ao analisarmos o custo global, temos a soma de n inserções, onde cada uma envolve uma busca de custo log n e um deslocamento de custo proporcional a n. Na notação assintótica, o termo de maior grau domina a expressão final, resultando em uma complexidade

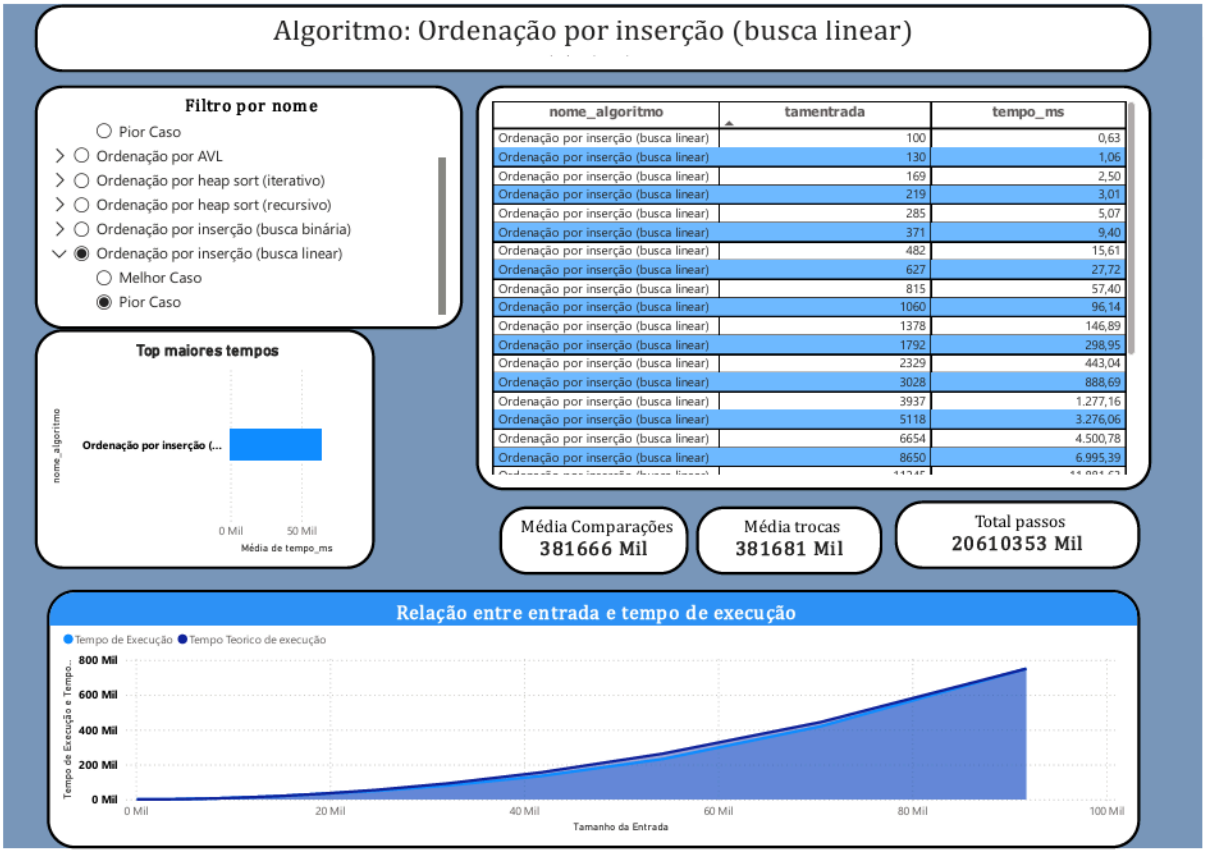
total de $\Theta(n^2)$. Assim, a otimização da busca melhora apenas o número de comparações realizadas, mas não altera a natureza quadrática do tempo de execução do **Insertion-Sort** devido à limitação física da estrutura de array.

Questão 05

No algoritmo **Heapsort**, o funcionamento pode ser analisado a partir de dois laços principais, sendo o mais importante, para a ordenação, o laço das linhas 4 a 7, responsável por posicionar corretamente os maiores elementos do vetor. O invariante de laço desse processo indica que, no início de cada iteração do laço, o subvetor A[i+1..n] encontra-se ordenado em ordem crescente e contém exatamente os maiores elementos do vetor original, todos já em suas posições finais corretas, enquanto o subvetor A[1..i] mantém a propriedade de heap máximo.
Na inicialização, após a construção do heap máximo nas linhas 1 e 2, nenhuma iteração do laço de ordenação foi realizada, logo i = n e o subvetor A[i+1..n] é vazio. De forma trivial, um vetor vazio está ordenado e não viola o invariante. Além disso, todo o vetor A[1..n] satisfaz a propriedade de heap máximo, garantindo que o maior elemento encontra-se na raiz, em A[1].
Na manutenção, assume-se que, no início de uma iteração qualquer i, o subvetor A[i+1..n] está ordenado e contém os maiores elementos do vetor, e que A[1..i] é um heap máximo. O algoritmo então troca A[1], que contém o maior elemento do heap, com A[i], colocando esse maior elemento exatamente em sua posição final correta. Em seguida, o tamanho do heap é reduzido para $i-1$, descartando-se o elemento já ordenado. Como a troca pode violar a propriedade de heap máximo na raiz, o procedimento **HEAP-DECREASE-KEY** é chamado a partir do índice 1, restaurando a propriedade de heap máximo em A[1..i−1]. Dessa forma, ao final da iteração, o subvetor A[i..n] permanece ordenado e o invariante é preservado para a próxima iteração.
No término, o laço encerra quando i = 2. Nesse momento, o subvetor A[3..n] já está ordenado e contém os maiores elementos, e restam apenas dois elementos em A[1] e A[2], que, pela propriedade do heap, já se encontram em ordem correta após a última troca. Assim, todo o vetor A[1..n] encontra-se ordenado, comprovando a correção do algoritmo **Heapsort**.
Quanto à análise de complexidade, a construção do heap máximo ocorre nas linhas 1 e 2 e possui custo $\Theta(n)$, pois os ajustes realizados nos nós internos do heap, quando analisados em conjunto, somam um tempo linear. Na fase de ordenação, o laço principal executa $n-1$ iterações e, em cada uma delas, o procedimento **HEAP-DECREASE-KEY** pode percorrer a altura do heap, que é $\Theta(\log n)$. Assim, cada iteração possui custo $\Theta(\log n)$, resultando em um custo total $\Theta(n \log n)$ para essa fase.
No melhor caso, mesmo que o vetor inicial já esteja ordenado, o algoritmo ainda constrói o heap máximo e executa todos os ajustes necessários após cada remoção da raiz. Portanto, o tempo de execução no melhor caso permanece $\Theta(n \log n)$. No pior caso, quando o vetor está em ordem completamente desfavorável, o comportamento do algoritmo não se altera, pois o **Heapsort** independe da ordem inicial dos elementos, mantendo também um tempo de execução $\Theta(n \log n)$.
Dessa forma, conclui-se que o algoritmo **Heapsort** apresenta complexidade de tempo $\Theta(n \log n)$ no melhor, médio e pior caso, sendo sua eficiência garantida pela manutenção da propriedade de heap máximo ao longo de todo o processo de ordenação.

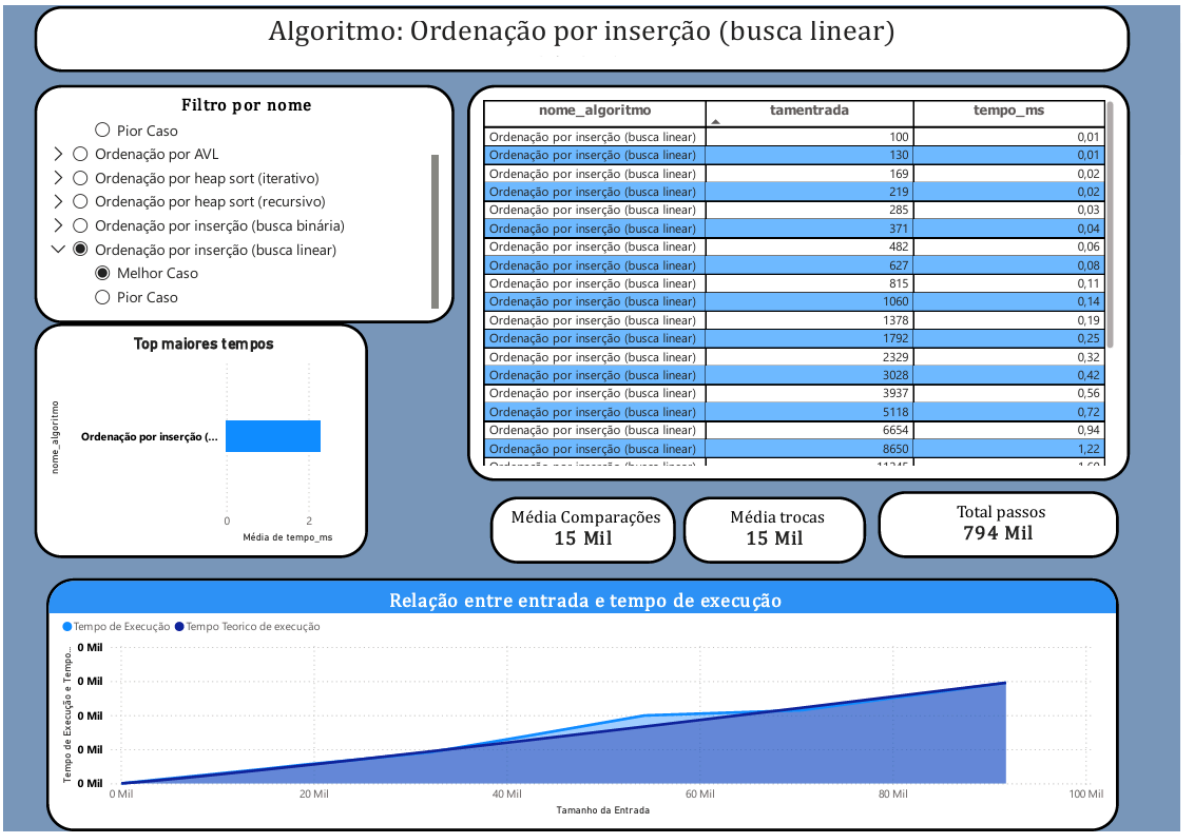
Questão 06

A análise dos resultados obtidos indica que o comportamento prático dos algoritmos de ordenação está, em grande parte, de acordo com as previsões teóricas. O Insertion-Sort com busca linear apresentou bom desempenho em vetores já ordenados ou quase ordenados, confirmando o melhor caso linear. Entretanto, em entradas aleatórias ou inversamente ordenadas, observou-se crescimento quadrático no número de comparações e movimentações, o que resultou em tempos de execução elevados.



nome_algoritmo	tamentrada	tempo_ms
Ordenação por inserção (busca linear)	100	0,63
Ordenação por inserção (busca linear)	130	1,06
Ordenação por inserção (busca linear)	169	2,50
Ordenação por inserção (busca linear)	219	3,01
Ordenação por inserção (busca linear)	285	5,07
Ordenação por inserção (busca linear)	371	9,40
Ordenação por inserção (busca linear)	482	15,61
Ordenação por inserção (busca linear)	627	27,72
Ordenação por inserção (busca linear)	815	57,40
Ordenação por inserção (busca linear)	1060	96,14
Ordenação por inserção (busca linear)	1378	146,89
Ordenação por inserção (busca linear)	1792	298,95
Ordenação por inserção (busca linear)	2329	443,04
Ordenação por inserção (busca linear)	3028	888,69
Ordenação por inserção (busca linear)	3937	1.277,16
Ordenação por inserção (busca linear)	5118	3.276,06
Ordenação por inserção (busca linear)	6654	4.500,78
Ordenação por inserção (busca linear)	8650	6.995,39

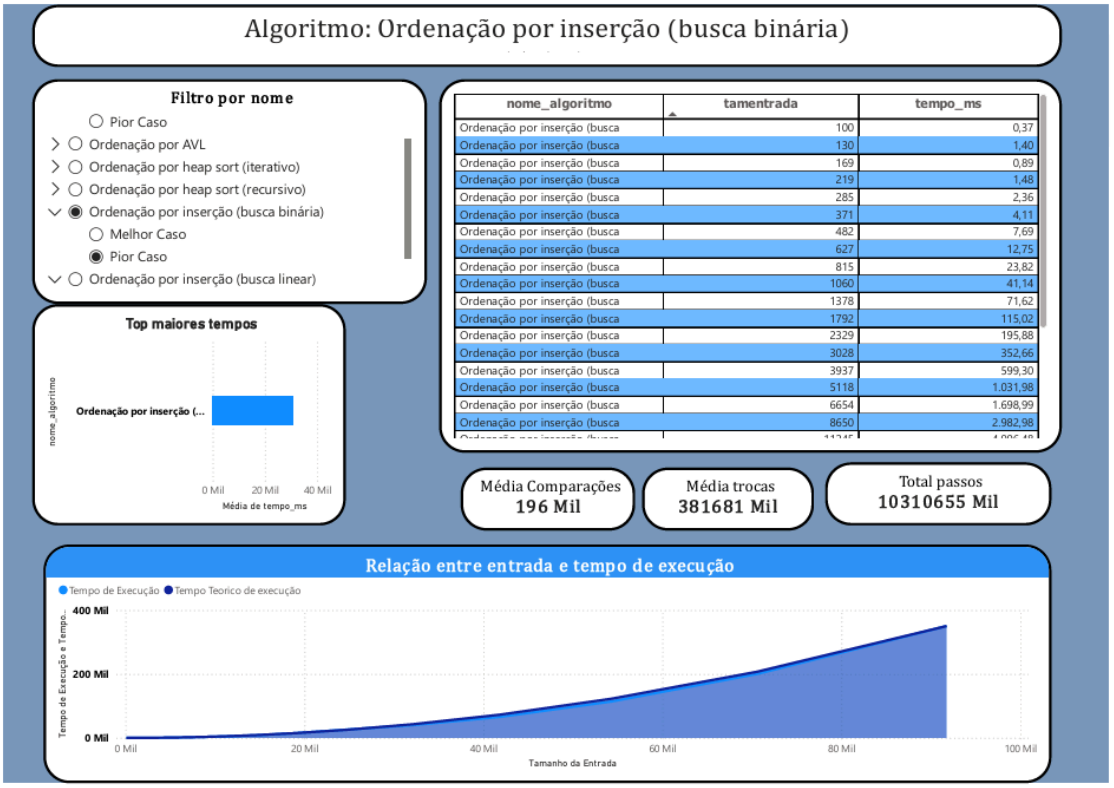
fig.1 pior caso. O gráfico azul claro representa a medida obtida no benchmark e o gráfico azul escuro representa a medida teórica da complexidade



nome_algoritmo	tamentrada	tempo_ms
Ordenação por inserção (busca linear)	100	0,01
Ordenação por inserção (busca linear)	130	0,01
Ordenação por inserção (busca linear)	169	0,02
Ordenação por inserção (busca linear)	219	0,02
Ordenação por inserção (busca linear)	285	0,03
Ordenação por inserção (busca linear)	371	0,04
Ordenação por inserção (busca linear)	482	0,06
Ordenação por inserção (busca linear)	627	0,08
Ordenação por inserção (busca linear)	815	0,11
Ordenação por inserção (busca linear)	1060	0,14
Ordenação por inserção (busca linear)	1378	0,19
Ordenação por inserção (busca linear)	1792	0,25
Ordenação por inserção (busca linear)	2329	0,32
Ordenação por inserção (busca linear)	3028	0,42
Ordenação por inserção (busca linear)	3937	0,56
Ordenação por inserção (busca linear)	5118	0,72
Ordenação por inserção (busca linear)	6654	0,94
Ordenação por inserção (busca linear)	8650	1,22

fig.2: o gráfico representa o melhor caso da ordenação por inserção

A utilização da busca binária reduziu o número de comparações, aproximando esse custo de O(n log n), mas não eliminou o custo quadrático total devido à necessidade de deslocamento dos elementos no vetor.



nome_algoritmo	tamentrada	tempo_ms
Ordenação por inserção (busca	100	0,37
Ordenação por inserção (busca	130	1,40
Ordenação por inserção (busca	169	0,89
Ordenação por inserção (busca	219	1,48
Ordenação por inserção (busca	285	2,36
Ordenação por inserção (busca	371	4,11
Ordenação por inserção (busca	482	7,69
Ordenação por inserção (busca	627	12,75
Ordenação por inserção (busca	815	23,82
Ordenação por inserção (busca	1060	41,14
Ordenação por inserção (busca	1378	71,62
Ordenação por inserção (busca	1792	115,02
Ordenação por inserção (busca	2329	195,88
Ordenação por inserção (busca	3028	352,66
Ordenação por inserção (busca	3937	599,30
Ordenação por inserção (busca	5118	1.031,98
Ordenação por inserção (busca	6654	1.698,99
Ordenação por inserção (busca	8650	2.982,98

fig.1. pior caso da ordenação por inserção utilizando busca binária

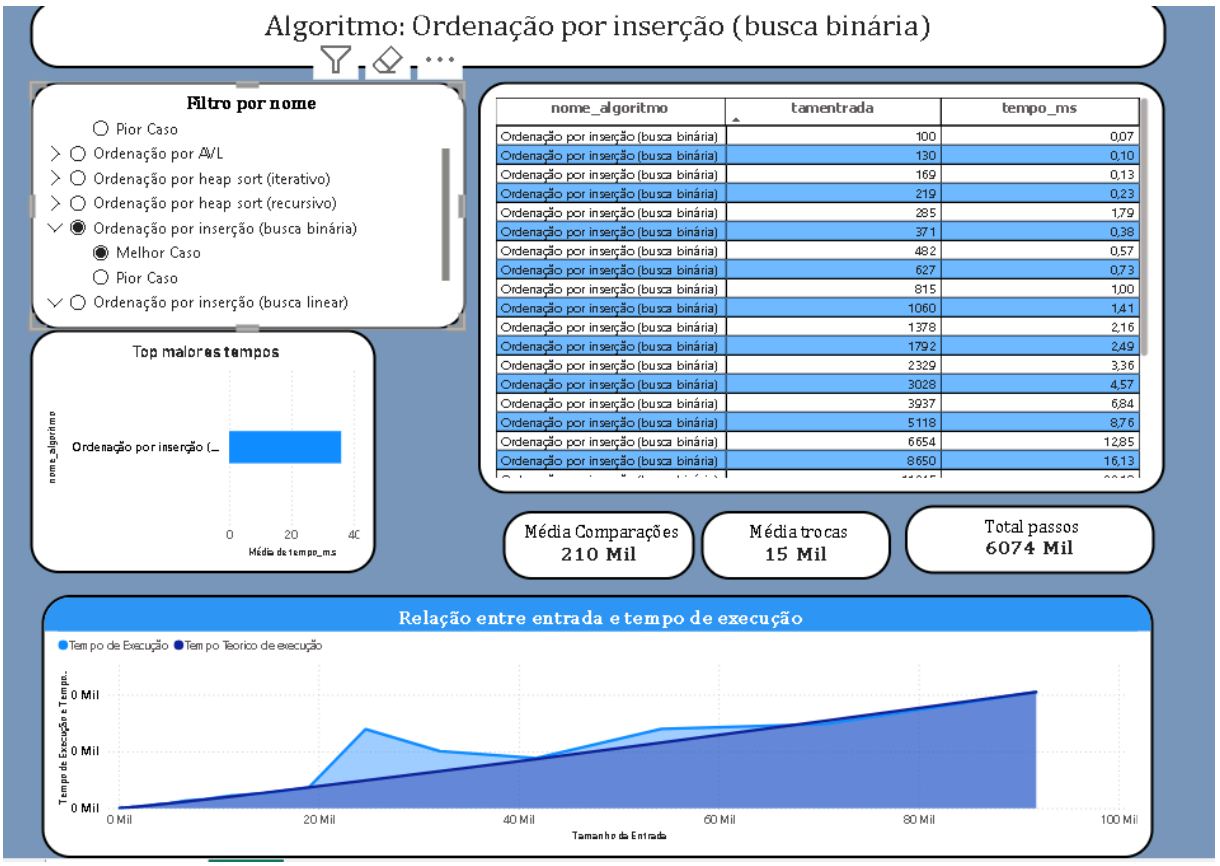
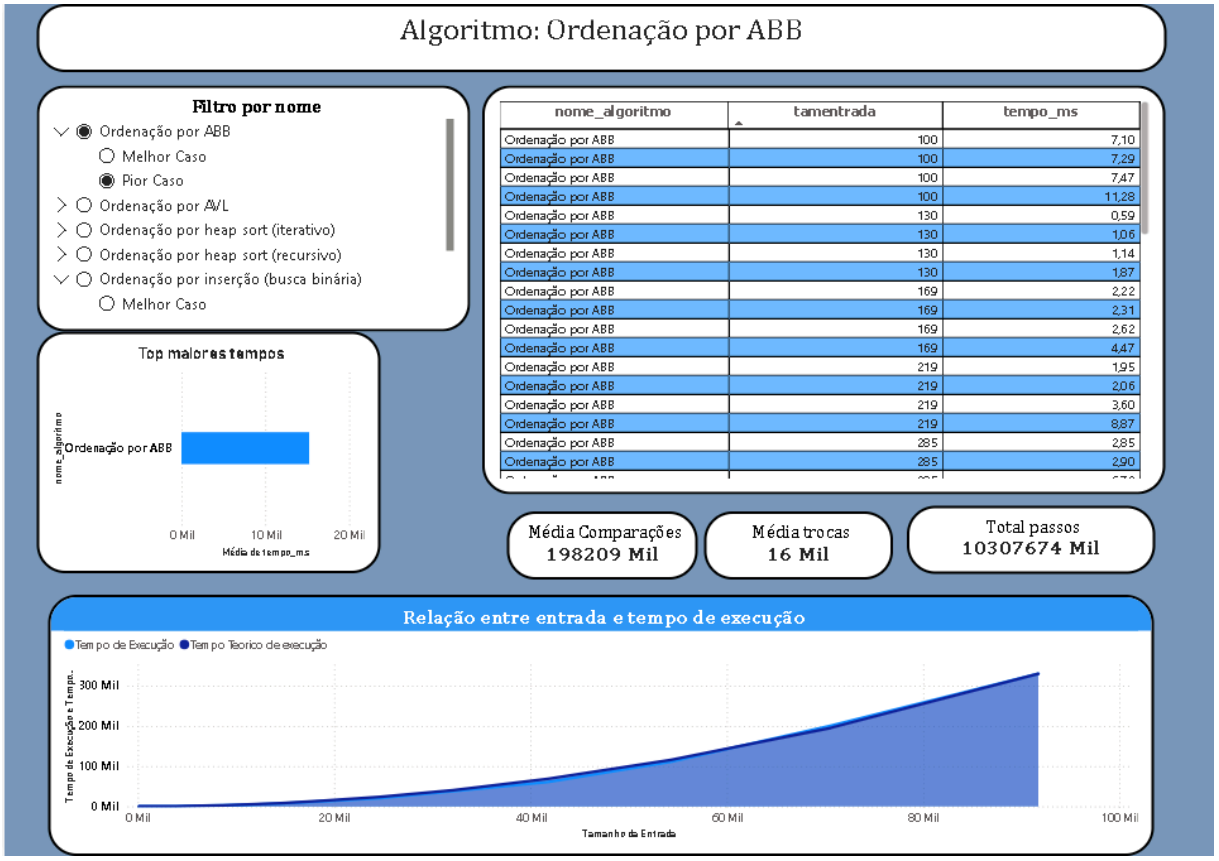


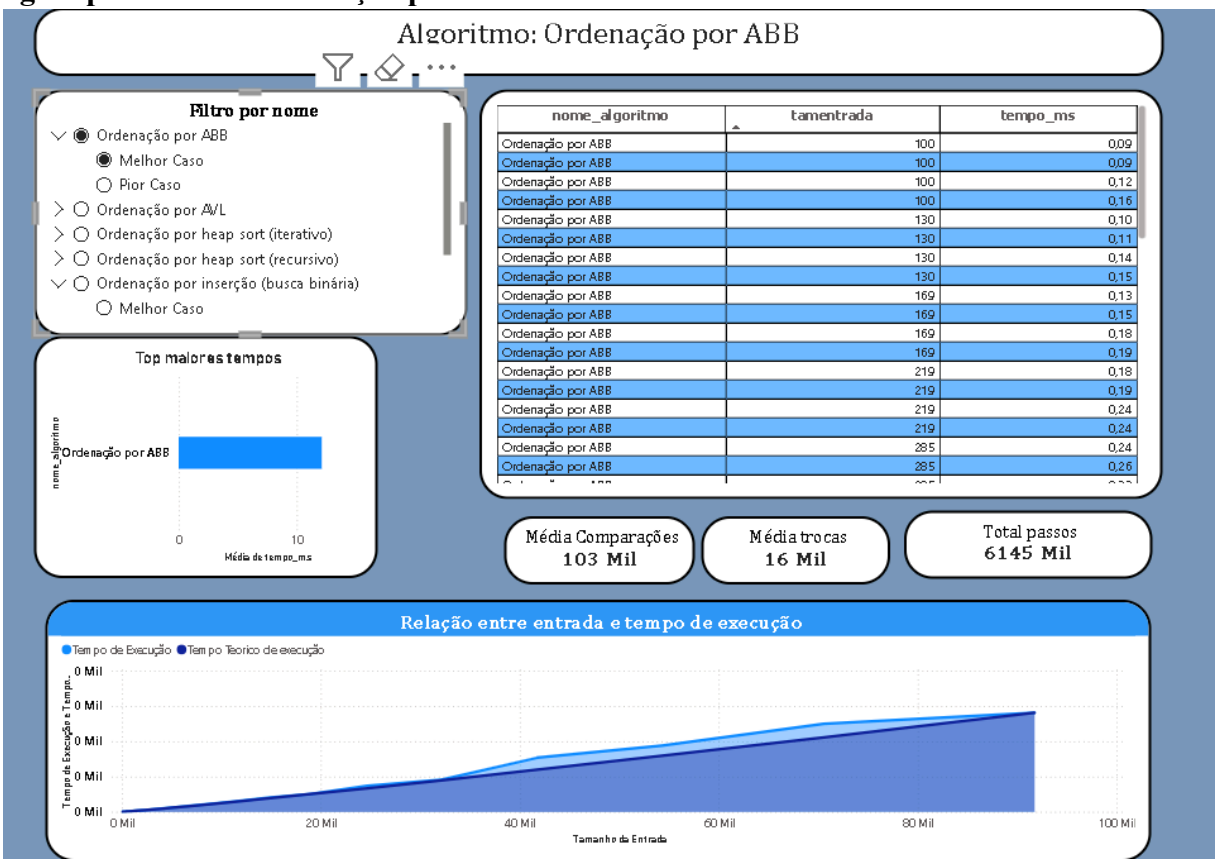
fig.2: melhor caso da ordenação por inserção utilizando busca binária

Na ordenação baseada em árvore binária de busca não balanceada, o desempenho mostrou-se fortemente dependente da ordem de inserção. Para dados aleatórios, o comportamento aproximou-se de O(n log n), enquanto entradas ordenadas provocaram a degeneração da árvore e elevaram o custo para $O(n^2)$, conforme previsto teoricamente.



nome_algoritmo	tamentrada	tempo_ms
Ordenação por ABB	100	7,10
Ordenação por ABB	100	7,29
Ordenação por ABB	100	7,47
Ordenação por ABB	100	11,28
Ordenação por ABB	130	0,59
Ordenação por ABB	130	1,06
Ordenação por ABB	130	1,14
Ordenação por ABB	130	1,87
Ordenação por ABB	169	2,22
Ordenação por ABB	169	2,31
Ordenação por ABB	169	2,62
Ordenação por ABB	169	4,47
Ordenação por ABB	219	1,95
Ordenação por ABB	219	2,06
Ordenação por ABB	219	3,60
Ordenação por ABB	219	8,87
Ordenação por ABB	285	2,85
Ordenação por ABB	285	2,90

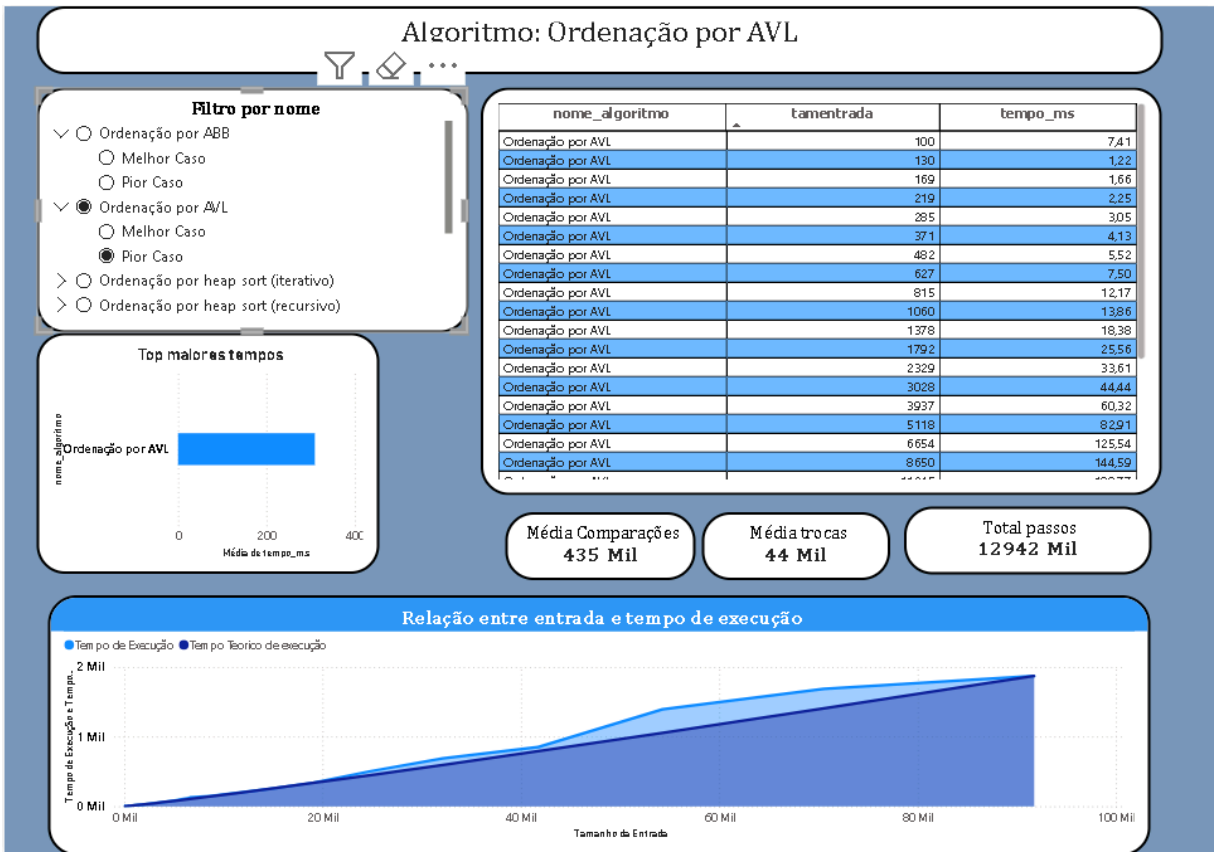
fig.1 : pior caso de ordenação por árvore binária de busca



nome_algoritmo	tamentrada	tempo_ms
Ordenação por ABB	100	0,09
Ordenação por ABB	100	0,09
Ordenação por ABB	100	0,12
Ordenação por ABB	100	0,16
Ordenação por ABB	130	0,10
Ordenação por ABB	130	0,11
Ordenação por ABB	130	0,14
Ordenação por ABB	130	0,15
Ordenação por ABB	169	0,13
Ordenação por ABB	169	0,15
Ordenação por ABB	169	0,18
Ordenação por ABB	169	0,19
Ordenação por ABB	219	0,18
Ordenação por ABB	219	0,19
Ordenação por ABB	219	0,24
Ordenação por ABB	219	0,24
Ordenação por ABB	285	0,24
Ordenação por ABB	285	0,26

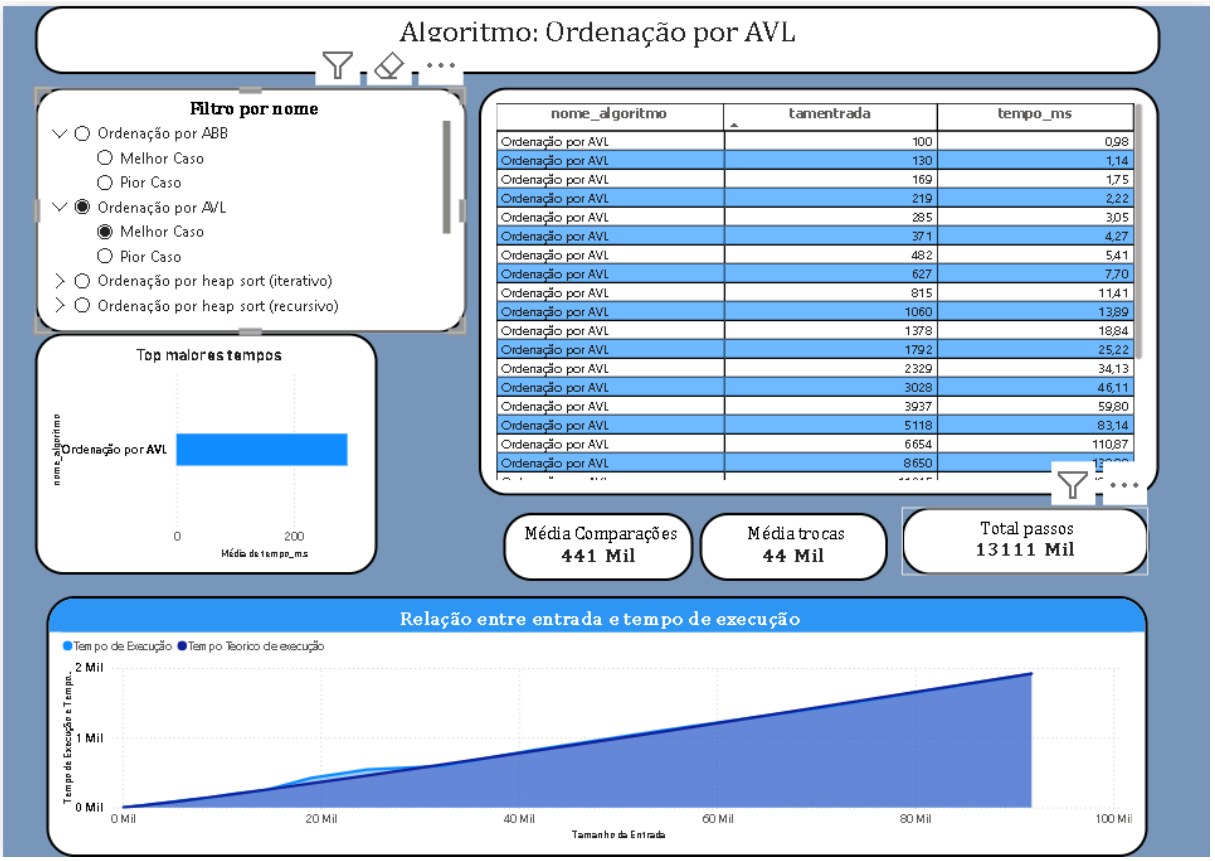
fig.2: melhor caso da árvore binária de busca

O uso de árvores balanceadas eliminou esse problema ao garantir altura logarítmica, resultando em desempenho mais estável, apesar do pequeno aumento de custo constante associado ao balanceamento.



nome_algoritmo	tamentrada	tempo_ms
Ordenação por AVL	100	7,41
Ordenação por AVL	130	1,22
Ordenação por AVL	169	1,66
Ordenação por AVL	219	2,25
Ordenação por AVL	285	3,05
Ordenação por AVL	371	4,13
Ordenação por AVL	482	5,52
Ordenação por AVL	627	7,50
Ordenação por AVL	815	12,17
Ordenação por AVL	1060	13,86
Ordenação por AVL	1378	18,38
Ordenação por AVL	1792	25,56
Ordenação por AVL	2329	33,61
Ordenação por AVL	3028	44,44
Ordenação por AVL	3937	60,32
Ordenação por AVL	5118	82,91
Ordenação por AVL	6654	125,54
Ordenação por AVL	8650	144,59

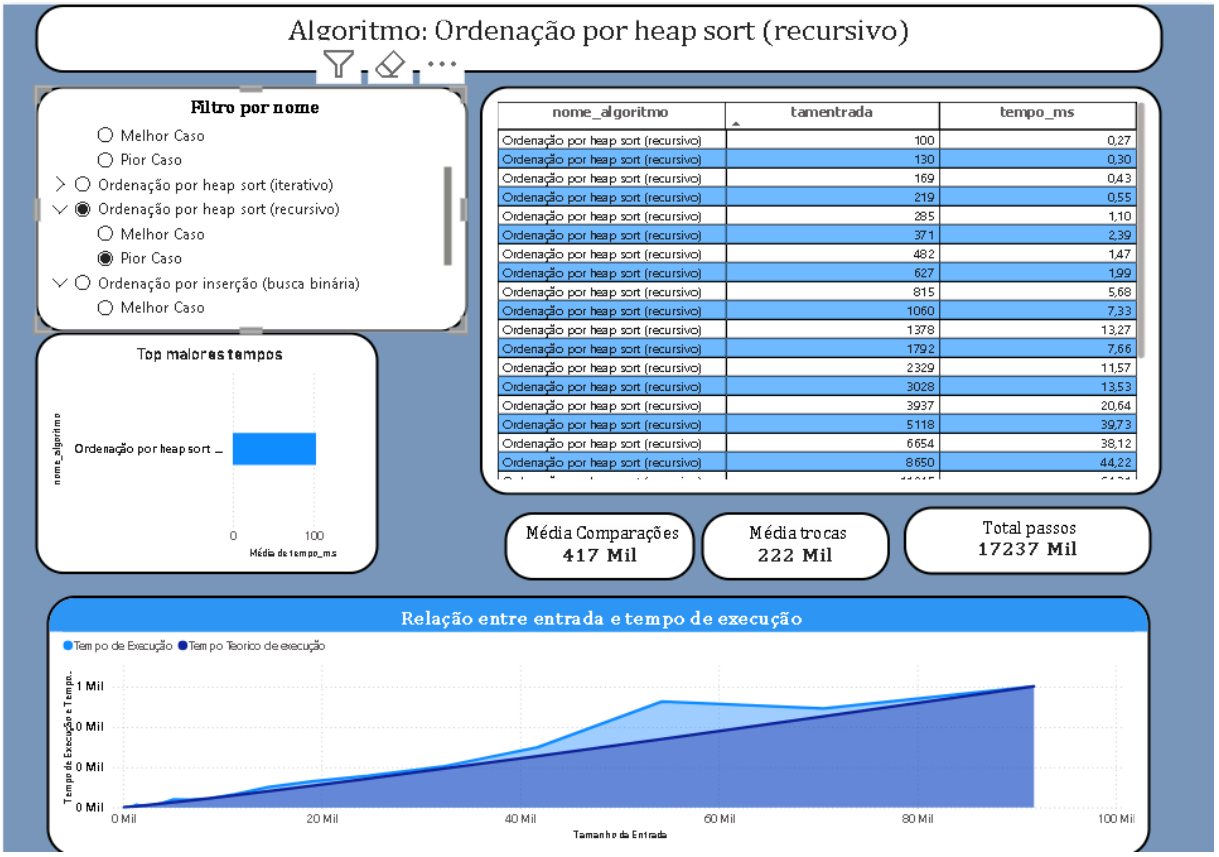
fig.1: pior caso da ordenação por AVL



nome_algoritmo	tamentrada	tempo_ms
Ordenação por AVL	100	0,98
Ordenação por AVL	130	1,14
Ordenação por AVL	169	1,75
Ordenação por AVL	219	2,22
Ordenação por AVL	285	3,05
Ordenação por AVL	371	4,27
Ordenação por AVL	482	5,41
Ordenação por AVL	627	7,70
Ordenação por AVL	815	11,41
Ordenação por AVL	1060	13,89
Ordenação por AVL	1378	18,84
Ordenação por AVL	1792	25,22
Ordenação por AVL	2329	34,13
Ordenação por AVL	3028	46,11
Ordenação por AVL	3937	59,80
Ordenação por AVL	5118	83,14
Ordenação por AVL	6654	110,87
Ordenação por AVL	8650	[illegible]

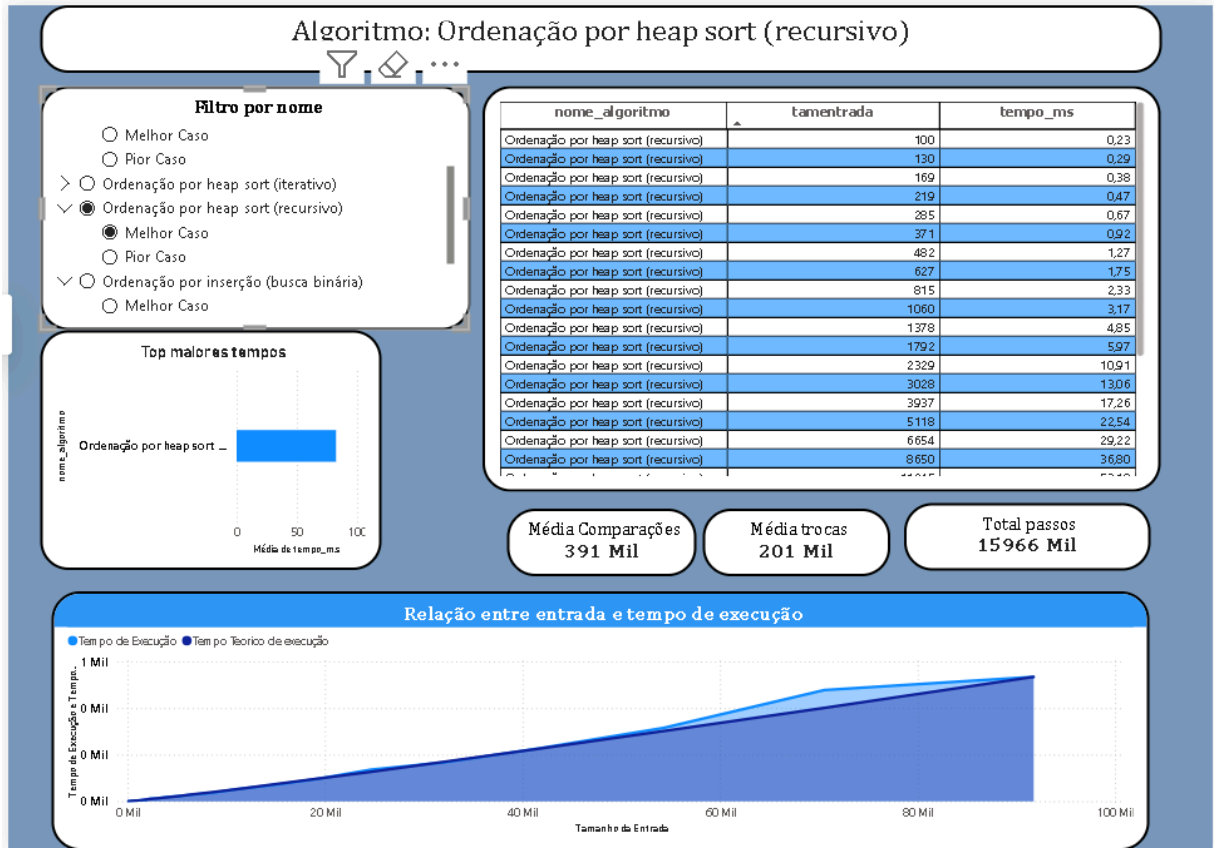
fig.2: melhor caso da ordenação por AVL

O Heap-Sort, em suas versões iterativa e recursiva, apresentou desempenho consistente em todas as massas de teste, mantendo tempo de execução próximo de O(n log n), independentemente da ordem inicial dos dados. A versão recursiva mostrou leve sobrecarga, sem impacto significativo na complexidade.



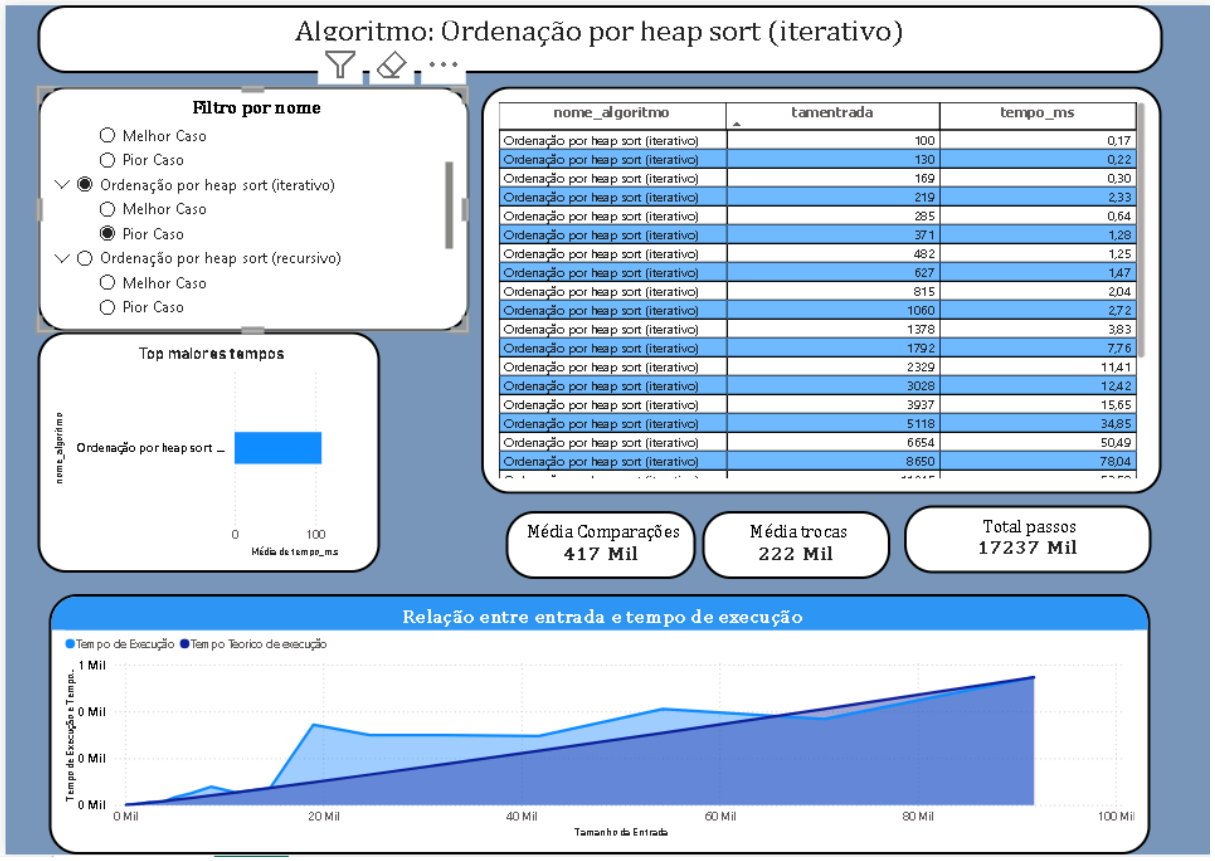
nome_algoritmo	tamentrada	tempo_ms
Ordenação por heap sort (recursivo)	100	0,27
Ordenação por heap sort (recursivo)	130	0,30
Ordenação por heap sort (recursivo)	169	0,43
Ordenação por heap sort (recursivo)	219	0,55
Ordenação por heap sort (recursivo)	285	1,10
Ordenação por heap sort (recursivo)	371	2,39
Ordenação por heap sort (recursivo)	482	1,47
Ordenação por heap sort (recursivo)	627	1,99
Ordenação por heap sort (recursivo)	815	5,68
Ordenação por heap sort (recursivo)	1060	7,33
Ordenação por heap sort (recursivo)	1378	13,27
Ordenação por heap sort (recursivo)	1792	7,66
Ordenação por heap sort (recursivo)	2329	11,57
Ordenação por heap sort (recursivo)	3028	13,53
Ordenação por heap sort (recursivo)	3937	20,64
Ordenação por heap sort (recursivo)	5118	39,73
Ordenação por heap sort (recursivo)	6654	38,12
Ordenação por heap sort (recursivo)	8650	44,22

fig.1: pior caso da ordenação por heapsort recursivo



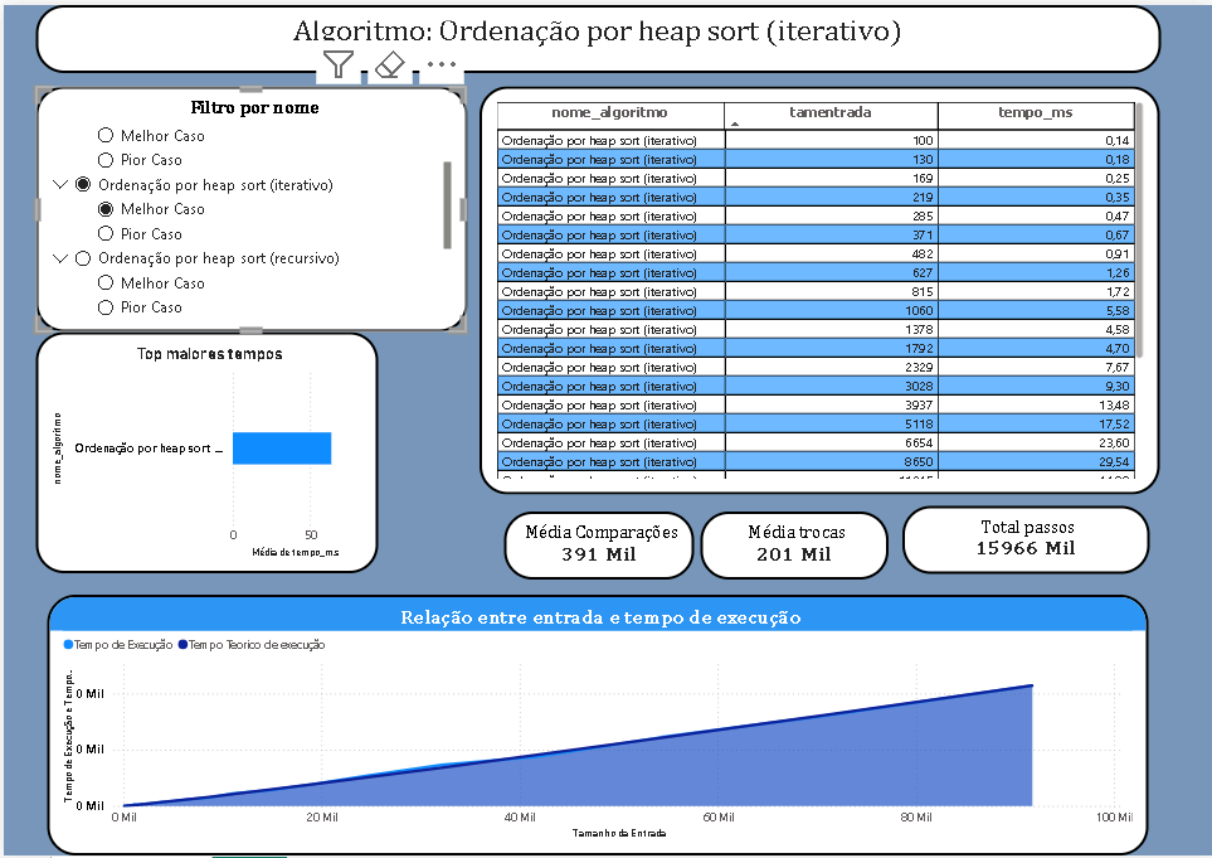
nome_algoritmo	tamentrada	tempo_ms
Ordenação por heap sort (recursivo)	100	0,23
Ordenação por heap sort (recursivo)	130	0,29
Ordenação por heap sort (recursivo)	169	0,38
Ordenação por heap sort (recursivo)	219	0,47
Ordenação por heap sort (recursivo)	285	0,67
Ordenação por heap sort (recursivo)	371	0,92
Ordenação por heap sort (recursivo)	482	1,27
Ordenação por heap sort (recursivo)	627	1,75
Ordenação por heap sort (recursivo)	815	2,33
Ordenação por heap sort (recursivo)	1060	3,17
Ordenação por heap sort (recursivo)	1378	4,85
Ordenação por heap sort (recursivo)	1792	5,97
Ordenação por heap sort (recursivo)	2329	10,91
Ordenação por heap sort (recursivo)	3028	13,06
Ordenação por heap sort (recursivo)	3937	17,26
Ordenação por heap sort (recursivo)	5118	22,54
Ordenação por heap sort (recursivo)	6654	29,22
Ordenação por heap sort (recursivo)	8650	36,80

fig.2:Melhor caso da ordenação por heap sort recursivo



nome_algoritmo	tamentrada	tempo_ms
Ordenação por heap sort (iterativo)	100	0,17
Ordenação por heap sort (iterativo)	130	0,22
Ordenação por heap sort (iterativo)	169	0,30
Ordenação por heap sort (iterativo)	219	2,33
Ordenação por heap sort (iterativo)	285	0,64
Ordenação por heap sort (iterativo)	371	1,28
Ordenação por heap sort (iterativo)	482	1,25
Ordenação por heap sort (iterativo)	627	1,47
Ordenação por heap sort (iterativo)	815	2,04
Ordenação por heap sort (iterativo)	1060	2,72
Ordenação por heap sort (iterativo)	1378	3,83
Ordenação por heap sort (iterativo)	1792	7,76
Ordenação por heap sort (iterativo)	2329	11,41
Ordenação por heap sort (iterativo)	3028	12,42
Ordenação por heap sort (iterativo)	3937	15,65
Ordenação por heap sort (iterativo)	5118	34,85
Ordenação por heap sort (iterativo)	6654	50,49
Ordenação por heap sort (iterativo)	8650	78,04

fig.1: pior caso da ordenação por heap sort iterativo



nome_algoritmo	tamentrada	tempo_ms
Ordenação por heap sort (iterativo)	100	0,14
Ordenação por heap sort (iterativo)	130	0,18
Ordenação por heap sort (iterativo)	169	0,25
Ordenação por heap sort (iterativo)	219	0,35
Ordenação por heap sort (iterativo)	285	0,47
Ordenação por heap sort (iterativo)	371	0,67
Ordenação por heap sort (iterativo)	482	0,91
Ordenação por heap sort (iterativo)	627	1,26
Ordenação por heap sort (iterativo)	815	1,72
Ordenação por heap sort (iterativo)	1060	5,58
Ordenação por heap sort (iterativo)	1378	4,58
Ordenação por heap sort (iterativo)	1792	4,70
Ordenação por heap sort (iterativo)	2329	7,67
Ordenação por heap sort (iterativo)	3028	9,30
Ordenação por heap sort (iterativo)	3937	13,48
Ordenação por heap sort (iterativo)	5118	17,52
Ordenação por heap sort (iterativo)	6654	23,60
Ordenação por heap sort (iterativo)	8650	29,54

fig.2: melhor caso da ordenação por heap sort iterativo

De forma geral, os resultados práticos confirmam a teoria, evidenciando que algoritmos simples são adequados apenas para cenários específicos, enquanto estruturas mais robustas garantem desempenho previsível em casos adversos.

Para mais detalhes do código e tabelas, acessar o GitHub projeto em: https://github.com/reali-705/analise-algoritmos.